Unit 5: Data Analysis Using Python Libraries

5.1 Introduction to Data Analysis and Visualization

Data Analysis is the systematic process of applying logical techniques to describe, condense, and evaluate data. Its importance across fields includes:

- Education: Identifying student learning patterns and predicting dropout rates.
- Healthcare: Analyzing patient records to predict disease outbreaks.
- Business: Understanding consumer behavior to increase "Return on Investment" (ROI).
- Visualization: This is the graphical representation of information. It is crucial because the human brain processes visuals 60,000 times faster than text, making complex Big Data patterns easy to spot.

5.2 Python Libraries and the Big Data Environment

Python is the preferred language for Big Data due to its massive ecosystem.

- The Environment: Analysts typically use Jupyter Notebooks or Google Colab. These environments allow for "literate programming," where code, output, and visualizations are kept in one place.
- Modular Roles: Instead of writing complex code from scratch, we use "Libraries" (pre-written code) to handle massive datasets efficiently.

5.3 NumPy and Statistical Functions

NumPy (Numerical Python) is the foundation for almost all scientific computing in Python.

- Key Concept: The ndarray (N-dimensional array). Unlike Python lists, NumPy arrays are stored in contiguous memory, making them incredibly fast for Big Data.
- Statistical Functions:
    - np.mean(): The arithmetic average.
    - np.median(): The middle value (useful when data has outliers).
    - np.std(): Standard Deviation (measures how spread out the numbers are).

5.4 Pandas for Data Handling (The ETL Workhorse)

Pandas is built on top of NumPy and provides the DataFrame—a 2D table-like structure with labeled axes (rows and columns).

Core Preprocessing Tasks:

1. Cleaning Missing Values: Using df.isnull() to find gaps, and df.fillna() or df.dropna() to fix them.
2. Handling Duplicates: Identifying and removing redundant entries.
3. Data Aggregation: Using the groupby() function to summarize data.
    - *Example:* Finding the Average Salary (Fact) per Department (Dimension).

5.5 Matplotlib for Data Visualization

Matplotlib is the standard library for creating static, high-quality graphs.

- Line Plots: Perfect for Time-Series data (e.g., stock prices or annual enrollment).
- Bar Charts: Ideal for Categorical comparisons (e.g., number of students per major).
- Scatter Plots: Used for Correlation (e.g., does studying more hours lead to higher grades?).

5.6 Data Ingestion and Exploration

In Big Data, the first step is getting the data into Python and understanding its "shape."

- Ingestion: Loading data from external sources.
    - df = pd.read_csv('data.csv')
    - df = pd.read_json('data.json')
- Exploration (The "First Look"):
    - df.head(): View the first 5 rows to understand the content.
    - df.info(): Check for data types and non-null counts.
    - df.describe(): Get a full statistical summary (min, max, mean, percentiles) in one command.

"Load a CSV, clean it, and find the mean," write this:

```
import pandas as pd
import numpy as np

# 1. Ingestion
df = pd.read_csv('students.csv')

# 2. Preprocessing
df.dropna(inplace=True) # Remove missing values

# 3. Exploration/Analysis
avg_score = np.mean(df['score'])
print(f"The average student score is: {avg_score}")
```